logiBUS® Mini-Trainer – Trainingskarten-Generator

Automatisierte Generierung von Schablonen und Trainingskarten für den **logiBUS® Mini-Trainer** aus Eclipse 4diac™ Subapplication-Dateien (`.SUB`).

Die Karten werden im **Hochformat (7,0 cm × 9,0 cm)** gerendert und passen exakt auf die Hardware-Schablone des Trainers (layout.png).

---

1. Übersicht & Konzept

Für Übungen in der Automatisierungstechnik (IEC 61499) werden Karten auf den Trainer gelegt. Die Signalleitungen führen von den physischen Eingangsklemmen links über die Logikbausteine zu den Ausgangsklemmen rechts.

- **Kartenmaße:** 70 mm Breite × 90 mm Höhe (Hochformat).
- **Eingänge (Links):** I_1 bis I_8 im 1-cm-Raster von oben (bei 1,0 cm, 2,0 cm, … bis 8,0 cm).
- **Ausgänge (Rechts):** Q_1 bis Q_8 im 1-cm-Raster von oben (bei 1,0 cm, 2,0 cm, … bis 8,0 cm).
- **Halbkreis-Klemmen am Rand:** Die Ein- und Ausgänge sind als Halbkreise direkt an die Außenkante gesetzt.
- **Dynamische Filterung:** Es werden ausschließlich die Pins dargestellt, die in der jeweiligen Übung auch verschaltet sind (z. B. nur I_1, I_2 und Q_1).
- **IEC 61499 Baustein-Layout:** Die I/O-Treiberbausteine (`logiBUS_IXA` , `logiBUS_QXA`) werden ausgeblendet; die Kernbausteine (z. B. `AX_AND_2`) werden im Vektor-Layout gerendert.
- **TGL-Normschrift:** Bausteine und Beschriftungen nutzen die originale, schmale Schriftart nach DDR-Norm `TGL 0-17_std` und `TGL 0-16_std` .
- **Textblock unten:** Zentrierter Übungsname und mehrzeilige Aufgabenbeschreibung über die vollen 7 cm Kartenbreite.

---

2. Erzeugte Ordnerstruktur & Ausgabe-Varianten

> **Fertige Karten herunterladen:** Die Karten werden nicht im Repository versioniert (`output/` steht in `.gitignore`). Die GitHub Action `.github/workflows/karten.yml` erzeugt sie mit `--batch-all` und veröffentlicht sie als Release: **`karten-latest`** (aktueller Stand von `main`) bzw. ein festes Release je Tag `v*` . Enthalten sind je Quellordner ein ZIP mit allen Varianten, die Sammelbögen (`*_all_exercises_multi.pdf`), `fix-TODO.md` und `QUELLEN.txt` mit den verwendeten Commits. In Pull Requests liegen die Sammelbögen als Artefakt am Workflow-Lauf.

Alle Dateien werden standardmäßig im Ordner `output/` getrennt nach Quellordner (`test_AX` und `test_B` , analog zum Workspace in Training1) mit Unterordnern je Typ abgelegt:

```
output/
├── test_AX/                # 497 Übungen aus test_AX
│   ├── exact_70x90/        # Exaktes 70×90-mm-Format ohne Ränder für Online-
Druckportale
│   │   └── <Uebung>_exact_70x90.pdf
│   ├── multi/              # Sammelbögen (2×3-Raster, 6 Karten/Seite auf DIN-A4)
│   │   ├── <Uebung>_multi.pdf
│   │   └── test_AX_all_exercises_multi.pdf  (bei Aufruf mit --all / --batch-all)
│   ├── printshop/          # Professionelle Druckerei-PDFs (Offset mit
TrimBox/BleedBox)
│   │   └── <Uebung>_printshop.pdf
│   ├── single/             # Einzelkarten zentriert auf DIN-A4 mit Schnittmarken
│   │   └── <Uebung>_single.pdf
│   └── svg/                # Standalone-Vektorgrafiken mit Base64-TGL-Schrift
│       └── <Uebung>_card.svg
└── test_B/                 # 374 Übungen aus test_B
    ├── exact_70x90/
    ├── multi/
    │   └── test_B_all_exercises_multi.pdf
    ├── printshop/
    ├── single/
    └── svg/
```

Unterordner	Dateinamensmuster	Format / Maße	Einsatzzweck
`<Ordner>/single/`	`<Uebung>_single.pdf`	DIN-A4	Schneller Kontrollausdruck (1 Karte mittig mit Schnittmarken und Beschreibung)
`<Ordner>/multi/`	`<Uebung>_multi.pdf`	DIN-A4	Papiersparender Selbstdruck (6 Karten pro A4-Seite im 2×3-Raster zum Ausschneiden)
`<Ordner>/printshop/`	`<Uebung>_printshop.pdf`	82 × 102 mm (Trim: 70×90 mm)	**Auftrags-Druckerei** mit Schnittmarken außerhalb des Papiers, Bleed- und TrimBox
`<Ordner>/exact_70x90/`	`<Uebung>_exact_70x90.pdf`	70 × 90 mm	Für Online-Druckereien, die Dateien 1:1 auf Endmaß ohne Schnittmarken fordern
`<Ordner>/svg/`	`<Uebung>_card.svg`	70 × 90 mm	SVG-Grafik mit Base64-eingebetteter TGL-Normschrift

---

3. Voraussetzungen

Für die Ausführung wird Python 3.10 oder neuer benötigt sowie folgende Python-Bibliotheken:

```
pip install reportlab svglib
```

Die Schriftarten (`TGL 0-17_std.ttf` , `TGL 0-16_std.ttf`) liegen in diesem Repository im Ordner `tgl/` . Der IEC 61499 Renderer liegt im Repository `IEC61499_FunctionBlock_SVG` , die Übungen und die Typbibliothek in `4diac_training1` . Beide werden standardmäßig neben diesem Repository erwartet; andere Orte lassen sich über Umgebungsvariablen angeben:

Variable	Standard	Inhalt
`IEC61499_SVG_TOOL`	`../IEC61499_FunctionBlock_SVG`	IEC 61499 Renderer
`TRAINING1_WORKSPACE`	`../4diac_training1/Ventilsteuerung/4diacIDE-workspace`	4diac-Workspace mit Übungen und `.lib`

4. Bedienung

Aufruf über die Batch-Datei (Windows)

Am einfachsten ist der Aufruf über die mitgelieferte Batch-Datei `generate_cards.bat`:

1. **Per Doppelklick:** Startet die Standard-Generierung (`Uebung_002a_AX.SUB`) und legt alle Varianten in `output/` ab.
2. **Über die Konsole mit Parametern:**

```
generate_cards.bat --sub "C:\git\ms-docs\4diac_training1\Ventilsteuerung\4diacIDE-
workspace\test_AX\Uebungen\Uebung_002a3_AX.SUB"
```

Aufruf über Python

```
# 1. Standard-Aufruf (erzeugt alle Varianten fuer Uebung_002a_AX in output/)
python generate_cards.py

# 2. Spezifische Uebungs-Datei uebergeben
python generate_cards.py --sub "C:\git\ms-docs\4diac_training1\Ventilsteuerung\4diacIDE-
workspace\test_AX\Uebungen\Uebung_002a5b_AX.SUB"

# 3. Alle Uebungen in ein mehrseitiges Druck-PDF (all_exercises_multi.pdf) generieren
python generate_cards.py --all

# 4. Alle Uebungen vollstaendig einzeln in alle Unterordner generieren
python generate_cards.py --batch-all

# 5. Testlauf fuer z.B. nur die ersten 6 Uebungen limitieren
python generate_cards.py --all --limit 6

# 6. Eigene Ausgabepfade oder anderes Basisverzeichnis angeben
python generate_cards.py --output-dir my_cards --sub Uebung_002a_AX.SUB
# 7. Layout-Analyse & fix-TODO.md
python generate_cards.py --check-layout
```

5. CLI-Parameter Referenz

Parameter	Typ	Standardwert	Beschreibung
`--sub`	Pfad	`Uebung_002a_AX.SUB`	Pfad zur 4diac-SubApp-Datei (`.SUB`)
`--output-dir`	Pfad	`output`	Basis-Ausgabeordner mit Unterordnern je Typ
`--all`	Flag	`False`	Mehrseitigen Sammelbogen für alle `.SUB` -Dateien erzeugen + `fix-TODO.md`
`--batch-all`	Flag	`False`	Alle Karten-Varianten für jede Übung im Ordner erzeugen + `fix-TODO.md`
`--check-layout`	Flag	`False`	Führt nur die Layout-Analyse durch und aktualisiert `fix-TODO.md`
`--todo-out`	Pfad	`fix-TODO.md`	Ausgabepfad für die Layout-TODO-Checkliste
`--limit`	Zahl	`None`	Maximale Anzahl an Übungen (z. B. `6` für Testläufe)
`--folder`	Pfad	`.../test_AX/Uebungen`	Quellverzeichnis für den `--all` -Modus
`--single-out`	Pfad	`output/single/...`	Individueller Ausgabepfad für Einzelkarte (DIN-A4)
`--multi-out`	Pfad	`output/multi/...`	Individueller Ausgabepfad für Sammelbogen (DIN-A4)
`--printshop-out`	Pfad	`output/printshop/...`	Individueller Ausgabepfad für Druckerei-PDF (Offset mit TrimBox)
`--exact-out`	Pfad	`output/exact_70x90/...`	Individueller Ausgabepfad für exaktes 70×90-mm-PDF
`--svg-out`	Pfad	`output/svg/...`	Individueller Ausgabepfad für Standalone-SVG

6. Automatische Layout-Prüfung & `fix-TODO.md`

Auf der physischen Trainingskarte liegen die digitalen Eingänge I_1 bis I_8 fest von oben (1,0 cm) nach unten (8,0 cm), und die Ausgänge Q_1 bis Q_8 rechts von 1,0 cm bis 8,0 cm. Wenn in 4diac ein Eingang mit höherer Nummer (z. B. I_3) weiter oben gezeichnet ist als ein kleinerer (z. B. I_1), müssen die Leitungen unschön über Kreuz führen.

Der Generator analysiert daher automatisch die Koordinaten aller Bausteine und Anschlüsse:

- **Pin-Inversionen:** Erkennt, wenn I_3 im 4diac-Netzwerk über I_1 gezeichnet wurde.

- **TODO-Checkliste:** Speichert alle betroffenen Übungen mit präzisen Handlungsempfehlungen in `fix-TODO.md`. Nach dem Verschieben der Bausteine in 4diacIDE wird das Kartenlayout beim nächsten Durchlauf automatisch sauber und kreuzungsfrei gerendert.

> [!TIP] Die Datei `aufgaben.json` dient als externe Aufgaben-Referenzsammlung für Schulungsszenarien. Die Kartengenerierung selbst liest direkt die Metadaten, I/O-Pins und Netzwerke aus den Eclipse 4diac `.SUB`-Dateien aus.

---

7. Lizenz & Urheberrecht

Copyright © 2026 Meisterschulen am Ostbahnhof München.
Lizenziert unter der Eclipse Public License 2.0 (EPL-2.0).